

Name: Kanika Saini

Program: Btech CSE B16

SAP ID: 590034098

HDOC DAY 27-

Question 1

The program must:

- Take positive values of rows and columns, repeating input if necessary.
- Create a variable-length 2D integer array.
- Take input **column by column**.
- Display the array in **row-major order using only one loop and a linear index**.

Algorithm

1. Start.
2. Input `rows`.
3. If `rows <= 0`, ask again until a positive value is entered.
4. Input `cols`.
5. If `cols <= 0`, ask again until a positive value is entered.
6. Declare `int a[rows][cols]`.
7. Use an outer loop for columns.
8. Use an inner loop for rows.
9. Read each element as `a[i][j]`.
10. Run one loop from 0 to `rows*cols - 1`.
11. Convert the linear index into:
 - `Row = k / cols`
 - `Column = k % cols`
12. Print `a[k/cols][k%cols]`.
13. Stop.

C Program

```
#include <stdio.h>
```

```

int main()
{
    int rows, cols;

    /* Input and validation of rows */
    do
    {
        printf("Enter number of rows: ");
        scanf("%d", &rows);

        if (rows <= 0)
            printf("Rows must be positive. Enter again.\n");
    } while (rows <= 0);

    /* Input and validation of columns */
    do
    {
        printf("Enter number of columns: ");
        scanf("%d", &cols);

        if (cols <= 0)
            printf("Columns must be positive. Enter again.\n");
    } while (cols <= 0);

    /* Variable Length Array */
    int a[rows][cols];

    printf("\nEnter array elements column by column:\n");

    /* Column-major INPUT */
    for (int j = 0; j < cols; j++)
    {
        for (int i = 0; i < rows; i++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    printf("\nArray in row-major order:\n");

    /* Single loop with linear index */
    for (int k = 0; k < rows * cols; k++)
    {
        printf("%d ", a[k / cols][k % cols]);

        if ((k + 1) % cols == 0)
            printf("\n");
    }

    return 0;
}

```

Example

Suppose:

Rows = 2
Columns = 3

```
(kali㉿kali)-[~/Desktop]
$ cc day27.c

(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter number of rows: 2
Enter number of columns: 3

Enter array elements column by column:
a[0][0] = 1
a[1][0] = 2
a[0][1] = 3
a[1][1] = 4
a[0][2] = 5
a[1][2] = 6

Array in row-major order:
1 3 5
2 4 6
```

Question 2

The second question asks you to read elements into a **variable-length 2D array**, copy those elements into a **1D array in row-major order**, and then display both arrays.

Algorithm

1. Start.
2. Input number of rows and columns.
3. Declare a VLA `a[rows][cols]`.
4. Declare a 1D array `b[rows*cols]`.
5. Input all elements of the 2D array.
6. Initialize `k = 0`.
7. Traverse the 2D array row by row.
8. Store `a[i][j]` into `b[k]`.
9. Increment `k`.
10. Display the 2D array.
11. Display the 1D array.
12. Stop.

C Program

```
#include <stdio.h>
```

```

int main()
{
    int rows, cols;

    printf("Enter number of rows: ");
    scanf("%d", &rows);

    printf("Enter number of columns: ");
    scanf("%d", &cols);

    int a[rows][cols];
    int b[rows * cols];

    printf("\nEnter elements of the 2D array:\n");

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    /* Copy 2D array into 1D array */
    int k = 0;

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            b[k] = a[i][j];
            k++;
        }
    }

    printf("\n2D Array:\n");

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }

    printf("\n1D Array in row-major order:\n");

    for (int i = 0; i < rows * cols; i++)
    {
        printf("%d ", b[i]);
    }

    return 0;
}

```

Example Output

Enter number of rows: 2
Enter number of columns: 3

```
(kali㉿kali)-[~/Desktop]
$ cc day27b.c

(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter number of rows: 2
Enter number of columns: 3

Enter elements of the 2D array:
a[0][0] = 10
a[0][1] = 20
a[0][2] = 30
a[1][0] = 40
a[1][1] = 50
a[1][2] = 60

2D Array:
10 20 30
40 50 60

1D Array in row-major order:
10 20 30 40 50 60
```